Microservices: A definition of this new architectural term

Citation: J. Lewis and M. Fowler "Microservices: A definition of this new architectural term," 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

Paper Information: Definitions

Rating: ⭐ ⭐ ⭐ ⭐

Own Keywords: Background, DevOps, Domain Driven Design (DDD), Microservices, Migration, Monolith

Reading Progress: Third Pass

Introduction (What is the Paper about?)

- The authors wish to describe a new up-coming trend in 2014 towards Microservices
- The authors seek to describe what they have seen is typical for microservices
  - they explain the major ideas and principles of microservices

Methods

- the authors describe their observations and experiences in industry applying no further or special methodology

Scientific Benefit

- **Several useful definitions:**
  - **Component:** *"A Component is a unit of software that is independently replaceable and upgradeable."*
  - **Libraries**: *"We define libraries as components that are linked into a program and called using in-memory function calls."*
  - **Service:** *"Services are out-of-process components who communicate with a mechanism such as a web service request, or remote procedure call."*
    - are independently deployable - libraries not
  - **Microservices:** *"In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies."*
    - seeks to build applications as suites of services
    - services seek to be independently deployable and therefore scalable
    - each service may use different programming languages and technology stacks to fit their central business capability the best
- **Enterprise Applications often consists of three main parts**:
  1. User Interface (UI)
  2. Server-Side Application
  3. Database (DB)
- **Decomposition Strategy:**

- **Library Level:** an application consisting of a decomposition of different libraries also can only be deployed together
  - you cannot deploy a single library
- **Service Level:** the decomposition in unique callable services make calls between services more costly

- **Team Management:**
  - Conway's Law: *"Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure."*
    - this means the decomposition of teams and their communication structure will become mirrored in the resulting software architecture
  - **Splitting around technical Layers:** Management often focus to split large application according technology layer leading to UI Teams, Server-Side-Logic Teams and Database Teams
    - even simple changes can result in a multi-team project resulting in scattered logic
  - **Splitting around business capabilities:** when teams become decomposed around business capabilities where each team include the full range of skills (From UI specialists to DB specialists) allowing them to use the full software stack (UI, BL and DB) this will result according Conway's Law in a software which is as well organized around business capabilities
    - explicit separation which is enforced due to the usage of service components make its easier to keep team responsibilities boundaries clear
  - **DevOps (Project vs. Products):** Microservice experts preferring that a teams owns a software product over it's full lifetime
    - A project model on the contrary side aim to deliver a finished piece of software to a maintenance company; the actually dev-team is then closed
    - The Project notion brings developers more into the contact how their software behaves in the wild
- **Deployability:**
  - **Monoliths are built as one single deployable unit**
    - all logic for handling a request runs just in a single process
    - any (even small) changes involve building and deploying a new version of the entire application
    - people are facing frustrations with monoliths as applications are more deployed to the cloud
  - **Microservices are individually deployable**
    - Microservice adds an opportunity for more granular release planning
    - Can speed up release process
- **Monoliths are horizontally scalable** by deploying multiple instances of the monolith itself rather
  - scaling of single parts of the monolith is not possible
  -
- **Monoliths struggle over time to maintain a good modular structure**
  - using just the provided features of the used programming language to divide and structure the application
  - hard to keep changes being only limited to one module/place in code
    - often affects many places in the code
- **Communication Systems:**
  - Products may put stress in a significant smart communication mechanism itself defining complex algorithms for message routing, choreography, transformations and applying business rules
  - Microservice community favors the approach ***smart endpoints and dump pipes***. The pipes are just focusing on reliably transferring the message not applying any logic to it.
    - **Reason:** Microservices aim to be as decoupled and cohesive *(Single Responsible)* as possible
    - similar to the classical Unix sense for filters and pipes
  - Communication inside a monolith is often done via method invocation or function call

    - Issue in changing the communication pattern when migrating monoliths to microservices: Changing from chatty, fine-grained communication with a coarser-grained approach
- **Governance:**
  - Centralized Governance tend to fix whole software on a single technology stack whereas experiences shows that this approach is restricting as other problems would benefit when using a different technology stack
    - Microservice makes the choice free to choose for every business capability a suitable technology stack
  - When enforcing something is better to provide the tool with the right settings and everything instead of just enforcing it through rules written on paper
    - *Governance of Code* instead of *Governance of Paper*
- **Data Management:**
  - Problem that domain model might differ between systems or even within a system
    - same entity can be viewed from different perspectives
  - *Domain-Driven Design (DDD)* can be used to cleanly divide a complex domain set up into multiple bounded contexts
    - it can maps and define the relationships between DDD as well
    - natural correlation between bounded contexts and service decomposition
  - Monolithic systems prefer a single database for persistent data even enterprises may prefer one database across a range of applications
  - Microservice prefer letting each service manage it's own database or a subset of tables which only belong to one service
    - called *Polyglot Persistence*
- **Design for failure:**
  - A consequence of a microservice architecture is that applications need to be able to handle and tolerate whenever a service fails or is crashed
    - introduce additional complexity --> disadvantage
  - Microservice put lots of emphasis on real-time monitoring
- **Using microservice choreographie and event collaboration can lead to emergent behavior**

Challenges / Future Work

- **Microservices Architectural style is an important idea**
  - many organizations are moving towards it including Netflix, Amazon and much more
- **The authors aren't sure that Microservices are THE future of software architectures**
  - their experiences are so far positive
  - not enough time has passed *(it was in 2014)* to make a full judgement
  - consequences of such architectural decisions are visible several years later after you made them
  - can't assess how microservice architectures mature
- **Microservices making it more difficult to move code across service boundaries**
  - important to get boundaries right
  - involves coordination when interfaces are changing, layers of backwards compatibility and testing is made more complicated
- **Components should compose cleanly**
  - when for every feature a own component is created there is a mess of connection logic
    - might miss that when thinking from the perspective of simple, small components
- **Teams should be adequately skillful**
  - a poor team will always create a poor system
  - microservices aren't helping in this aspects
    - they are making the system not easier

- **It is recommended to build a microservices from the beginning not to start with monolith when possible**
  - a good in-process interface is usually not a good service interface

Limitations

- authors are active members of the microservice community itself
  - potentially more subjective in this regard

Paper Outline and Read Progress

1. [X] Introduction: Definition of microservices and how it is different from monoliths
2. [X] Characteristics of a Microservice Architecture*: Section Introduction*
3. [X] Componentization via Services
4. [X] Products not Projects *(DevOps)*
5. [X] Smart endpoints and dumb pipes
6. [X] Decentralized Governance
7. [X] Decentralized Data Management
8. [X] Infrastructure Automation
9. [X] Design for failure
10. [X] Evolutionary Design
11. [X] Are Microservices the Future?

Own Comments

- (+) provides several basic ideas and definitions
- (+) key article of microservices
- (-) statements are based on observations
- (-) Not peer-reviewed and no systematic evidence.
- (-) Potentially outdated; published in 2014.
- (-) No specific focus on legacy modernization, service identification, or ML-based migration.
- (-) Examples are strongly shaped by large tech companies; generalizability is limited.